

Towards Accelerating Physics Informed Graph Neural Network for Fluid Simulation

Yidi Wang
NVIDIA
Singapore
Singapore Institute of Technology
Singapore
jwangyidi@nvidia.com

Frank Guan
Singapore Institute of Technology
Singapore
frank.guan@singaporetech.edu.sg

Malcolm Yoke Hean Low
Singapore Institute of Technology
Singapore
malcolm.low@singaporetech.edu.sg

Zhengkui Wang
Singapore Institute of Technology
Singapore
zhengkui.Wang@singaporetech.edu.sg

Aik Beng Ng
NVIDIA
Singapore
aikbengn@nvidia.com

Simon See
NVIDIA
Singapore
ssee@nvidia.com

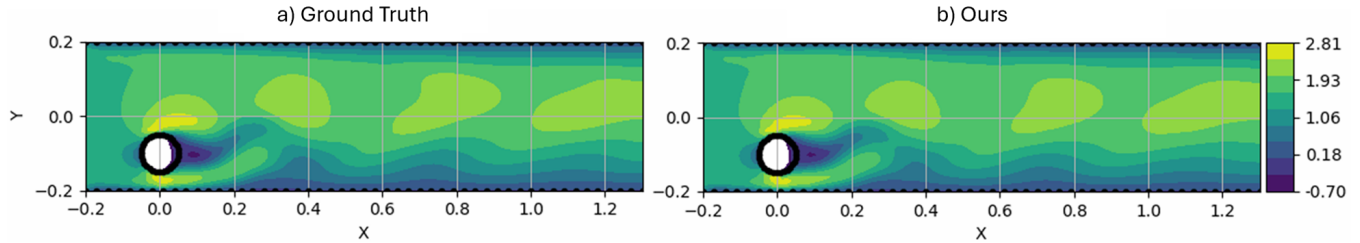


Figure 1: Representation of a snapshot of a cylinder flow test simulations. a) the ground truth, b) our simulation result. Circle at left is an obstacle. The color code represents the x component of the dimensionless velocity field.

Abstract

Fluid simulations in Computational Fluid Dynamics (CFD) often involve solving complex Partial Differential Equations (PDEs) with varying parameters, leading to high computational costs and slow inference times. Graph neural networks (GNNs) offer a promising alternative by efficiently representing mesh spaces and reducing these costs. In this work, we introduce a pioneering Physics-Informed Graph Neural Network (PIGNN) approach, utilizing a multi-GNN processor architecture that assigns each output parameter to a dedicated GNN processor with individual decoders. This innovative method reduces the training epochs for PIGNN by half while preserving accuracy. Additionally, we adopted mixed-precision training and achieved further improvements in reducing the training time to one-quarter of the original network while maintaining VRAM usage.

ACM Reference Format:

Yidi Wang, Frank Guan, Malcolm Yoke Hean Low, Zhengkui Wang, Aik Beng Ng, and Simon See. 2024. Towards Accelerating Physics Informed Graph Neural Network for Fluid Simulation. In *SIGGRAPH Asia 2024 Posters*

(SA Posters '24), December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3681756.3697879>

1 Introduction

Many challenges in fluid simulation, particularly in Computational Fluid Dynamics (CFD), involve repeatedly solving complex systems of partial differential equations (PDEs) with varying parameter values. Traditional PDE solvers in CFD, while known for their accuracy, often suffer from substantial computational costs and slow inference times, especially when dealing with complex geometries and intricate meshes [Jeon et al. 2024].

Graph neural networks (GNNs) offer a promising alternative for accelerating fluid simulations [Sanchez-Gonzalez et al. 2020]. GNNs can effectively capture information in the mesh grid of the simulation space. By leveraging the mesh structure, GNNs can provide efficient and scalable solutions, generalize across different parameter sets and geometries, and enhance their utility in practical applications requiring rapid and accurate simulations [Sanchez-Gonzalez et al. 2020]. This efficiency and generalization ability make GNNs a powerful tool for potentially reducing computational costs when solving PDEs and improving inference speeds in fluid simulations.

Physics-informed neural networks (PINNs) [Jeon et al. 2024] are also popular in fluid simulation. They require less data for training and can accelerate CFD simulations by approximating the solutions for PDE constraints. For example, in the seminal work of Hernández [Hernández et al. 2022], a Physics-Informed Graph Neural Network (PIGNN) was introduced for thermal fluid simulation. This network

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1138-1/24/12

<https://doi.org/10.1145/3681756.3697879>

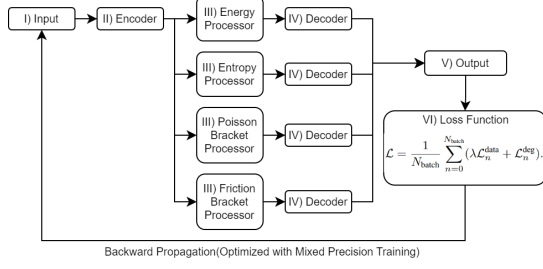


Figure 2: Our proposed network structure

leverages GNNs with soft constraints derived from the laws of energy conservation and entropy inequality, achieving relative mean errors of less than 3%. However, the training process for this network remains time-consuming.

2 Our Approach

2.1 Method

In order to accelerate physics-informed graph neural networks, we developed a novel multi-GNN processor strategy shown in Figure 2. In our method, each output in the network is assigned with its own lower latent space processor and each processor will conduct training in parallel. The output from each processor is further processed through individual decoders. Our approach reduces the training time significantly without altering the training variables, while maintaining the network’s accuracy.

Our input is a graph G_0 constructed from the initial state variables of the environment X_0 at time t_0 , where $G_0 = f(X_0, t_0)$. The vertex features v_i represent the state variables, and the edge features e_{ij} represent the relative distance between vertices, such that $G_0 = (e_{ij}, v_i)$. In our encoder, we employ two Multilayer Perceptrons (MLPs) to convert the initial vertex and edge feature vectors into higher-dimensional embeddings, where $e_{ij} \rightarrow x_{nij}$ and $v_i \rightarrow x_{ni}$, with $n = \{E, S, L, M\}$ representing each of the processors: Energy (E), Entropy (S), Poisson Bracket Matrix (L), and Friction Bracket Matrix (M). We employ four separate processors, each working on a single output parameter. Each processor is a GNN where it updates the edge feature with an MLP, and then updates the node feature with another MLP where $(x_{nij}, x_{ni}, x_{nj}) \rightarrow x'_{nij}$ and $(x_{ni}, \Phi(x_{nij})) \rightarrow x'_{ni}$, where Φ is an invariant pooling function. Subsequently, four decoding functions are employed, each decoding to a single output where $x'_{ni} \rightarrow y_{ni}$. Finally, the network will output new state variables X_1 at time t_1 , which will be used for loss calculation. The network backpropagates the loss and uses the state variable X_1 to predict the next frame of the simulation.

Additionally, we employed mixed-precision training (MP) to reduce memory consumption associated with multi-processor methods, further accelerating the training process. MP utilizes half precision (FP16) instead of higher precision formats such as FP32 or FP64 for weights, effectively reducing the memory footprint of neural networks and the time spent during training.

2.2 Experimental results

We compared our method with the reported method by Hernández [Hernández et al. 2022]. Our proposed method achieves a similar error rate but with 40% fewer epochs, thus reducing the training time by around half. However, the GPU memory cost of training the network increases with the use of multiple processors from 9gb to 14gb. We experimented with reducing the latent space dimension for each processor, as training for a single parameter might not necessitate a network as large as the original. However, our experiments revealed that the network’s convergence is highly sensitive to these adjustments, and decreasing the latent space dimension can negatively impact convergence.

To address this issue, we used MP techniques and cut GPU VRAM usage to one-third. However, mixed-precision training is affecting the network stability. We addressed this problem with loss scaling [NVIDIA 2023], which prevents gradient underflowing during backwards propagation. This further decreased training time by 70% comparing to the original network, allowing the network to converge to similar accuracy with even lesser training time. All networks are trained on a single NVIDIA A6000 GPU. The error shown in the table is an average of Velocity and Pressure prediction error.

Table 1: Experiment Results for Fluid Velocity Simulation

Network	Training Time (Hours)	VRAM Cost (GB)	Relative L2 Error
PIGNN	203	9	4.83
PIGNN + Multi Processor	125	14	4.99
PIGNN + Multi Processor + Mixed Precision	57	8	4.87

3 Conclusion and discussion

In this paper, we introduce a multi-processor PIGNN method that assigns each output parameter to its own GNN processor with individual decoders. This approach halves the training epochs needed for PIGNN while maintaining accuracy. We also employed mixed-precision training, which cuts our VRAM usage to two-thirds while reducing the training time to one quarter comparing to the original network. Our future work is to extend our network to 3D temperature field simulations.

Acknowledgments

This research is partially supported by MOE Translational R&D and Innovation Fund (MOE2023_TIF-0013).

References

- Quercus Hernández, Alberto Badias, Francisco Chinesta, and Elías Cueto. 2022. Thermodynamics-informed graph neural networks. *arXiv preprint arXiv:2203.01874* (2022).

Joongoo Jeon, Juhyeong Lee, Ricardo Vinuesa, and Sung Joong Kim. 2024. Residual-based physics-informed transfer learning: A hybrid method for accelerating long-term CFD simulations via deep learning. *International Journal of Heat and Mass Transfer* 220 (2024), 124900. <https://doi.org/10.1016/j.ijheatmasstransfer.2023.124900>

NVIDIA. 2023. Train with Mixed Precision. <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>. Accessed: 2024-07-27.

Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. 2020. Learning to Simulate Complex Physics with Graph Networks. In *International Conference on Machine Learning*. 8372–8382. Available: <https://arxiv.org/abs/2002.09405>.